

Lecture 1. Algorithm Analysis

1. 计算复杂性:

1) 记法:

O notation = $f(n) = O(g(n))$ iff.
upper bound

$\exists C, \exists n_0, \forall n > n_0, f(n) \leq Cg(n)$ 如果 $\forall C$, 都是 $<$
则是 $O(g(n))$

Ω notation = $f(n) = \Omega(g(n))$ iff.

lower bound

$\exists C, \exists n_0, \forall n > n_0, f(n) \geq Cg(n)$ 同理用 $\omega(g(n))$

Θ notation = $f(n) = \Theta(g(n))$ iff.

$f(n) = O(g(n)), f(n) = \Omega(g(n))$

2) 时间 complexity: 运行时间的复杂度

空间 complexity: 除去输入需要的 complexity

$\hookrightarrow S(n) = O(T(n))$ 开空间需要时间

2. 排序算法及其 $T(n)$

1) 选择排序

for $i \rightarrow 1$ to $n-1$

for $j \rightarrow i+1$ to n $T(n) = \text{best} = \text{worst} = O(N^2)$

if $A[i] > A[j]$ $S(n) = O(1)$

swap.

2) 冒泡排序.

while $i \leq n-1$,

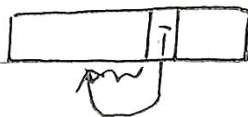
for $j \rightarrow n$ downto $i+1$ $T(n) = \text{best} = \text{worst} = O(N^2)$

if $A[j] < A[j-1]$ $S(n) = O(1)$

swap

$i++$.

(3) 插入排序:



for $i \rightarrow 2$ to n :

$x = A[i]$; $j = i - 1$

$T(n) = O(n^2)$

while $j > 0$ & $A[j] > x$.

$S(n) = O(1)$

$A[j+1] = A[j]$; $j--$

$A[j+1] = x$

(4) merge sort

if $l \geq r$, return

$m = \lfloor (l+r)/2 \rfloor$

$T(n) = O(n \log n)$

merge sort ($A[l, \dots, m]$)

merge sort ($A[m+1, \dots, r]$)

merge (A, l, m, r)

(5) quick sort.

$i = 1$ pivot = $A[n]$

for $j \rightarrow 1$ to $n-1$

$T(n) = n \log n$

if ($A[j] < \text{pivot}$)

→ 之后可去 PPT 上看计算

swap ($A[i], A[j]$)

$i = i + 1$

swap $A[i], A[n]$

if $i > 1$ qs ($A[1, \dots, i-1]$)

if $i < n$ qs ($A[i+1, \dots, n]$)

Lecture 2. Divide & Conquer.

1. Divide & Conquer.

(1) 策略:

- ① 把问题 P 分为若干 same type 的子问题
- ② recursively 解决这些子问题 $\Rightarrow$ key: 临界解
- ③ 合理 combine 这些 solved problem.

(2) 示例:

计算 n -bit 数的乘积 (n 为 power of 2)

$$x = \boxed{x_L} \boxed{x_R} = 2^{\frac{n}{2}} x_L + x_R$$

$$y = \boxed{y_L} \boxed{y_R} = 2^{\frac{n}{2}} y_L + y_R$$

$$xy = 2^n x_L y_L + 2^{\frac{n}{2}} (x_L y_R + x_R y_L) + x_R y_R \quad \text{divide}$$

化为 $(x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$

$$\text{从而 } T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

(3) 复杂度 main theorem (主定理)

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d), \quad a > 0, b > 1, d \geq 0$$

$$T(n) = \begin{cases} O(n^d), & \text{if } d > \log_b a \\ O(n^d \log n), & \text{if } d = \log_b a \\ O(n^{\log_b a}), & \text{if } d < \log_b a \end{cases}$$

2. 应用.

(1) binary search.

low = 1 high = n, j = 0
while (low ≤ high && j == 0)
mid = $\frac{1}{2}(l+h)$
if (x == A[mid]) j = mid.

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$\Rightarrow T(n) = \log n$$

if (x < A[mid]), high = mid - 1
else low = mid + 1

2) mergesort 把 $A[p, \dots, q]$ 与 $A[q+1, \dots, r]$ 有序的 merge

merge 算法: $s = p$; $t = q+1$; $k = p$

while $s \leq q$ & $t \leq r$:

if $A[s] \leq A[t]$

$B[k] = A[s]$, $s++$;

else $B[k] = A[t]$, $t++$;

$k++$;

if $s = q+1$ $B[k, \dots, r] = A[t, \dots, r]$

else $B[k, \dots, r] = A[s, \dots, q]$

mergesort 算法: (自底向上)

$t = 1$

while $t < n$

$s = t$, $t = 2s$, $i = 0$

while ($i + t \leq n$)

merge (A , $i+1$, $i+t$, $i+t$)

$i = i+t$

if ($i+t < n$)

merge (A , $i+1$, $i+t$, n)

3) 矩阵乘法,

见 PPT.

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$\rightarrow T(n) = O(n \log n)$$

$$D = \max\{d_x, d_y\} + 1 \text{ 初始为 } 0$$

3. Sorting Network

(1) 0-1 principle = 一个 sorting network 只要能够正确

给 0-1 输入排序, 也就设计成功

反证法, 若输入排 0, 1, 但有一个 a_i, a_j 排反.

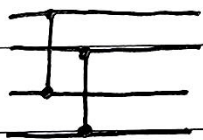
$$\text{定义 } f(x) = \begin{cases} 0 & x \leq a_i \\ 1 & x > a_i \end{cases} \Rightarrow f(a_i) = 0, f(a_j) = 1 \text{ 排反}$$

(2) 构建排序网:

① bitonic sorter

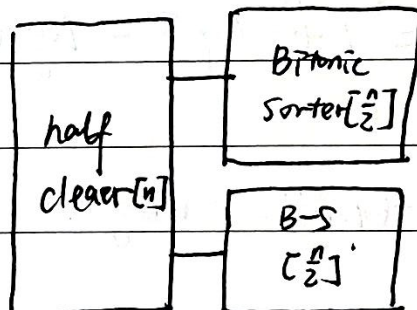
先增后减 or 循环往复 $0^k 1^k / 1^k 0^k$

half cleaner:



: 就能把一半变为 clean 的, 一半是 bitonic 的

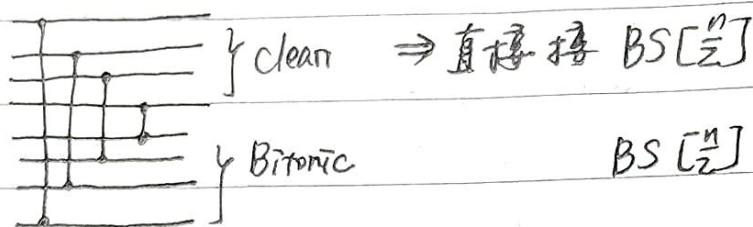
从而 bitonic sorter 可以这样



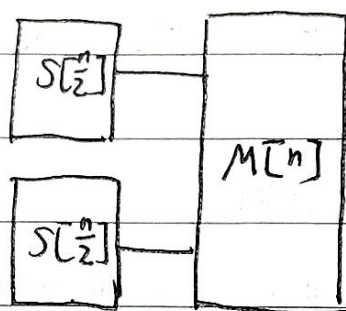
$$D(n) = D\left(\frac{n}{2}\right) + 1. \quad \approx O(\log n)$$

② merger.

→ 把下方的 倒过来看, 就是 bitonic 了



③ sorter



三个分治.

最小都是 $BS[2]$

复杂度:

① bitonic sorter:

$$D(n) = D(\frac{n}{2}) + 1 \Rightarrow D(n) = \log n$$

② merger (注意并行)

$$D(n) = 1 + \log \frac{n}{2} = \log n$$

③ sorter

$$D(n) = D(\frac{n}{2}) + D(\log n) \Rightarrow D(n) = \log^2 n$$

定理: $T(n) = aT(\frac{n}{b}) + O(n^d \log n)$

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } d > \log_b a \\ O(n^d \log^2 n) & \text{if } d = \log_b a \\ O(n^{\log_b a} \log n) & \text{if } d < \log_b a \end{cases}$$

Lecture 3. Greedy Algorithm

1. Basic Methodology:

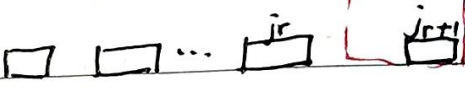
(1) 区间调度: 找出最多 compatible 的个数

① 贪心策略: 选结束时间最早的.

proof:

假设贪心不是最优, 取最多相同的那个最优.

greedy:  $\rightarrow$ 由于贪心结束最早

opt:  交换一下更好. 从而 $r+1$ 更大

与最大产生了 minimum contradiction $\Rightarrow$ greedy is opt.

2. 区间划分

① 贪心策略: 先尽量安排空教室, 若有冲突, 则开新的

def: $\text{depth} = \text{所有时间段最大冲突数} \Rightarrow \text{depth} \leq \text{opt}$

proof:

有 $A_g \geq A^* \geq \text{depth}$

而 Greedy 时开新教室就是因为 $d-1$ 冲突

从而 $A_g = \text{depth} \Rightarrow A_g = A^*$

2. 代码:

sort by starting time.

$d=0$, for $j \rightarrow n$.

if j 与 classroom k 可 $\rightarrow$ 维护一个堆, 取出最早结束的.

else $d=d+1$.

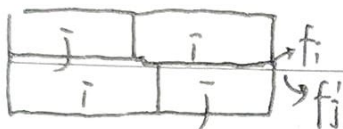
最小化延迟调度

→ 给定任务需要的时间 & due date

① 贪心策略: 按 deadline 来排序

proof:

一定有 (如果有一个)



考虑相邻逆序

$$l_j = f_j' - d_j$$

最后也是没 S^* 是最少 inversion $= f_i - d_j$

所以, 然后交换得更少

$$\leq f_i - d_i = l_i \quad \text{交换后只好不坏}$$

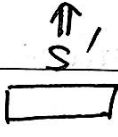
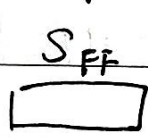
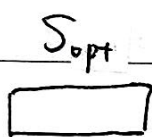
2. more example

1) offline Caching

① 贪心策略: Farther Future

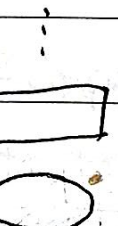
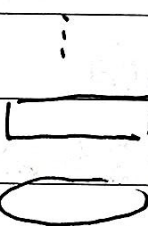
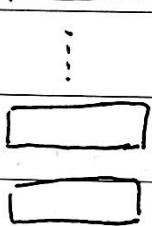
proof:

想要找一个 j opt



前 j 个 S 与 S_{FF} 一样

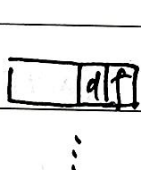
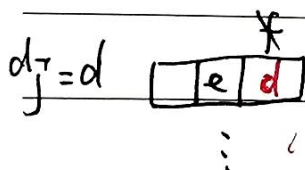
若 $j+1$ 时 d miss 了



$g=f$ 如果 S 去了 e , 那 S' 一样

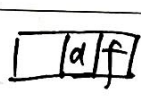
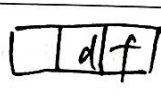
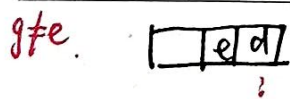
如果 S 去了 e' , 那 S' 将

e' 换成 e



$g \neq e, f$
 $g = g$ 则 g 换 $e \Rightarrow S'$ 换 f

一样



2) select breakpoints.

选跑得最远的点 (证明套路同上)

3) coin changing:

Greedy 成立条件:

$$C_{t+1} = pC_t - \delta \quad 0 \leq \delta < C_t, \quad C_t \leq C_{t+1}, \quad p \in \mathbb{Z}^+$$

那么 $G_{t+1}(y)$ is opt $\Leftrightarrow G_{t+1}(pC_t)$ is opt

$$\Leftrightarrow 1 + G(\delta) \leq p.$$

线性无关
不含环的边集
uniform matrix
余图连通

Lecture 4. Matroid 拟阵

1. Matroid 的概念.

(1) matroid is independent $\Leftrightarrow$ 继承性.

$$\downarrow \quad A \subset B, B \in C \Rightarrow A \in C \quad \text{显然中要是.}$$

(2) matroid is exchanged $\Rightarrow$ 贪心贪心

C 相当于可行值

$$\downarrow \quad A, B \in C \ \& \ |A| < |B| \Rightarrow \exists x \in B \setminus A. A \cup \{x\} \in C$$

$\downarrow$ 不一定是子集

2. Greedy Algorithm on Matroid

(1) $u(F)$ & $v(F)$

① 扩展: independent 的可否再吃新元素

$\rightarrow$ 极大的最小

$$u(F) = \min \{ |I| \mid I \text{ 是 } F \text{ 的极大独立集} \}$$

$$v(F) = \max \{ |I| \mid I \text{ 是 } F \text{ 的独立子集} \}$$

$\rightarrow$ 最大

贪心求最优解: $u(F) \leq v(F)$

建设性 交互性

B) 拟阵等价定理

$\Leftrightarrow$ 一个独立系统 (S, C) 是拟阵 $\Leftrightarrow$ 任意 $F \subseteq S$ $v(F) = u(F)$

$\Leftrightarrow$ 贪心 is opt.

$\rightarrow$ 称为 basis 基

$\rightarrow$ 如果是 min 则用 $c' = c_{max} - c$ 即可

3) Greedy-Max 算法

A. ① 先排序 $c(x_1) \geq c(x_2) \geq \dots \geq c(x_n)$

② $A = A \cup \{x_i\}$ (当 $A \cup \{x_i\} \in C$)
 $\rightarrow$ 独不独立

$O(n \log n + n f(n))$

B. Greedy theorem for 独立系统

速联系后 $\Leftarrow \star$ $1 \leq \frac{c(A^*)}{c(A_g)} \leq \max_{F \in S} \frac{v(F)}{u(F)}$

面的近似算法 证明可以去看 slides, 不会任意坏

C. 贪心最优 $\Leftrightarrow$ 是拟阵.

$\Leftarrow$ 见 B. $p=1$ 即可

$\Rightarrow$ 反证法, 如果不是拟阵

则有两个大小不一样的独立集. $|I| < |J|$
可以分配权重使得 Greedy is not opt.

3. 单位时间任务调度 ~~(按 deadline 排)~~ 把 penalty 大 w_i 的放前
 "1" $\rightarrow$ 安排 job (有 ddl & 逾期惩罚), 使 penalty 最小

1) 两步降维:

① early-first form:

如果逾期那就可以随便放后面

② Canonical Form:

如果都没逾期, 那把早 deadline 放前

$\rightarrow$ 问题简化为在 n 个任务中挑出来 early.

先排 w_i , 然后

$\rightarrow$ 一个一个看 ddl. $N_t(A) > t$ 就不要.
 不要

(K) Greedy Approach:

① 定义: 独立: 挑出来的一组 A , 没有逾期的.

$N_t(A)$: A 中 $ddl \leq t$ 的个数

② Lemma:

A 独立 $\Leftrightarrow N_t(A) \leq t \Leftrightarrow A$ 中任务按 ddl 排序不逾期.

1) $\Rightarrow$ 2) 显然

2) $\Rightarrow$ 3): 若 $d_i \leq i \Rightarrow N_{d_i}(A) \geq i > d_i$ 矛盾

3) $\Rightarrow$ 1): 显然

$O(n \log n + n^2) = O(n^2)$

③ 拟阵:

给 w_i 排序

$1 \leq t \leq k$

$N_t(A \cup \{x\}) = N_t(A) \leq t$

遗传性显然

$N_t(A \cup \{x\}) = N_t(A) + 1$

交换性: $|B| > |A|$; 我们考虑 $N_t(A) \geq N_t(B)$

$(k+1 \leq x \leq n) \leq N_t(B) \leq t$

中 t 最大的 k . 则 $k+1$ 时 $N_t(A) < N_t(B)$ 那 B 中有一个 $ddl = k+1$ 的, 可以放入 A

Lecture 5. Dynamic Programming

1. introduce examples.

(1) Weighted Interval Scheduling (有权)

解法: ① 定义 $p(j) = i$, 表示 i 是最大与 j compatible 的 index

② 定义 $OPT(j)$ 表示对 $1, 2, \dots, j$ 时最好解法

$$OPT(j) = \begin{cases} 0 & j=0 \\ \max \{ w_j + OPT(p(j)), OPT(j-1) \} \end{cases}$$

选 j 不选 j

直接递归是指数时间

优化:

① 备忘录法: 将已经做好的存起来

$O(n \log n)$

最多查 n 次 $\Rightarrow O(2n) = O(n)$ (两个例子都对)

更新后 $MT[j]$, 可回溯找出对应解

② 制表法 (Bottom-Up):

$$MT[j] = \max \{ w_j + \text{都叫 } j-1, OPT(j-1) \}$$

2. popular recipes.

(1) segmented least square

定义 trade-off function: $E + cL$, for constant $c > 0$

这是 multiway choice.

解法: ① 定义 $OPT(j) = \text{minimum cost for } p_1, p_2, \dots, p_j$

$e(i, j) = \text{minimum sum of square } p_i, p_{i+1}, \dots, p_j$

表示最后一条线段起点

$$OPT(j) = \begin{cases} 0 & j=0 \\ \min_{1 \leq i \leq j} \{ e(i, j) + c + OPT(i-1) \} & \text{otherwise} \end{cases}$$

优化: 初始算 e_{ij} ($j=1 \sim n, i=1 \sim j$ e_{ij} 又需要 $O(n^3)$)
 $\Rightarrow$ Time $O(n^3)$ Space $O(n^2)$

计算 e_{ij} 时, 可以预先算出 $\sum x_i, \sum y_i, \sum x_i y_i, \sum x_i^2$
 之后用前缀和 可以在 $O(1)$ 得出答案

(2) knapsack problem (背包问题)

解法: $opt(i, w) = \begin{cases} 0 & j = 0 \\ \text{对 } 1, 2, \dots, i, w \text{ 能} & \begin{cases} opt(i-1, w), & w_i > w \\ \max\{opt(i-1, w), v_i + opt(i-1, w-w_i)\} \end{cases} \end{cases}$

time complexity $O(nw)$ (为多项式 (NP))

(3) RNA second structure (区间 dp)

条件: ① $A \leftrightarrow U, C \leftrightarrow G$

② 如果 $b_i, b_j \in S$, then $i < j-4$

③ no crossing

只用一维时会自然切分, 无法形成子问题

$opt(i, j) = \begin{cases} 0, & i \geq j-4 \\ \max \begin{cases} opt(i, j-1), & i \leq j-4 \\ 1 + \max_t \{opt(i, t-1) + opt(t+1, j-1)\} \end{cases} \end{cases}$

进而引出填表问题 (要用到的数据之间的依赖关系) $\rightarrow$ 用左边界和下面

法1: 按子串的长皮 $k=j-i$ ($5 \sim n-1$)

法2: 顺序 i 去填

法3: 按 j 去序

(1,0)

$\begin{matrix} * & (1,9) \\ & i & j \end{matrix}$

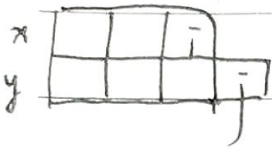
3. Hirschberg's Alignment Algorithm

(1) 编辑距离

$X = x_1 x_2 \dots x_m$ 与 $Y = y_1 y_2 \dots y_n$ 对齐 cost 最小

cost 分为 ① Gap 距离, 扣 δ

② mismatch 扣 $2x_i y_j$



解法:

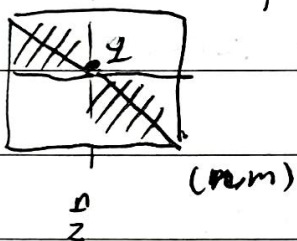
$$\text{opt}(i, j) = \begin{cases} \min \begin{cases} \text{opt}(i-1, j-1) + 2i_j \\ \text{opt}(i-1, j) + \delta \\ \text{opt}(i, j-1) + \delta \end{cases} \\ i\delta / j\delta, \quad j=0 / i=0 \end{cases}$$

时间 $O(mn)$, 空间 $O(mn)$ 爆炸

(2) 优化线性空间 complexity

解法 1: 用滚动数组去记录两列状态

解法 2: divide & conquer + DP.



定义前向 $f(i, j)$, 后向 $g(i, j)$

最小就为 $f(0, q) + g(q, n)$

$$T(m, n) \leq \underbrace{cmn}_{\text{算 } f, g} + T(q, \frac{n}{2}) + T(m-q, \frac{n}{2})$$

归纳可得 $T(mn) \leq 2cmn$.

空间: 每个 f, g 用滚动数组

$$\Rightarrow O(m+n)$$

数组 记录

$O(\log n)$ 递归

Lecture 6. Linear Programming

1. 线性 programming

(1) 定义 & standard

→ Both objective & constraints are Linear

标准形式:

$$c_1, c_2, \dots, c_n, \quad b_1, b_2, \dots, b_m, \quad a_{ij} \quad (i=1, 2, \dots, m, j=1, 2, \dots, n)$$

找出 $x_1, x_2, \dots, x_n$ s.t.

$$\begin{cases} \max & \sum c_j x_j \\ \text{s.t.} & \sum_{j=1}^n a_{ij} x_j \leq b_i \\ & x_j \geq 0 \end{cases}$$

→ 如果 min 就 max 反的

→ 如果 = 就用 $\geq$ & $\leq$, $\leq$ 再用反

→ 如果有不限制的用 $x = x^+ - x^-$ $x^+, x^- \geq 0$

(2) slack form:

限制条件只有 $x_j \geq 0$,

引入 slack variables, s.t. $\sum_{j=1}^n a_{ij} x_j + s_i = b_i, s_i \geq 0$

(3) Integer Linear Programming

同上一样, 只是变成了 $x_i \in \mathbb{N}$ (NP问题)

eg.

① $G = (V, E)$. 找最大独立集 $S \subseteq V$

对每个节点定义 $x_u = \begin{cases} 1, & u \text{ 被选入} \\ 0, & \text{未选入} \end{cases}$

即求 $\max \sum_{u \in V} x_u$

s.t. $x_u + x_v \leq 1$ (不相邻不入)

ILP 易出题.

② dominant set

将 bool 变量设在点/边上.

$G = (V, E)$ 找最小 $S \subseteq V$ 使得每个节点或 $v \in V$, 或有邻

点在 S 中 (骨子网络)

$$x_u = \begin{cases} 1 & u \in S \\ 0 & u \notin S \end{cases}$$

Neighbor function

$$\min \sum_{u \in V} x_u, \text{ s.t. } x_u + \sum_{v \in N(u)} x_v \geq 1, \forall u \in V$$

2. duality

(1) 形式:

$$\text{primal} \quad \max \sum_{j=1}^n c_j x_j$$

$$\text{s.t. } \sum_{j=1}^n a_{ij} x_j \leq b_i$$

$$x_j \geq 0$$

$$\text{dual} \quad \min \sum_{i=1}^m b_i y_i$$

$$\text{s.t. } \sum_{j=1}^n a_{ij} y_i \geq c_j$$

$$y_i \geq 0$$

(2) 例子:

$$\max x_1 + 6x_2$$

$$\text{s.t. } y_1 (x_1 + x_2) \leq 400$$

引入 primal 的系数 $y_2 (x_1) \leq 200$

$$y_3 (x_2) \leq 300$$

$$x_1, x_2 \geq 0$$

$$\min 400y_1 + 200y_2 + 300y_3$$

$$\text{s.t. } y_1 + y_2 \geq 1 \quad x_1$$

$$y_1 + y_3 \geq 6 \quad x_2$$

$$y_1, y_2, y_3 \geq 0$$

显然 dual 的解是 primary 的上界

如果存在 dual 与 primal 的目标值相等, 则两者都最优

3.

弱对偶定理:

$$\text{primal value} \leq \text{dual value}$$

强对偶定理:

非基本变量 (原始) 基本变量 (由 slack 约束)

3. 单纯形法 (simplex)

(1) 定义: LP 的可行域是个凸多面体时,
最优解一定存在于 Vertex 上.

迭代步骤 (e.g.)

① 有可能不会停

② 要是不是 tight 就不了

$$\max x_1 + 6x_2$$

$$\text{s.t. } x_1 + x_2 \leq 400$$

$$x_1 \leq 200$$

$$x_2 \leq 300$$

$$x_1, x_2 \geq 0$$

$$\max x_1 + 6x_2 \quad (0, 0, 400, 200, 300)$$

$$\text{s.t. } x_1 + x_2 + x_3 = 400 \quad (400 - x_1 - x_2 = x_3)$$

$$x_1 + x_4 = 200$$

$$x_1 + x_5 = 300$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0$$

显然提升 x_2 更好, 约束为

$$\begin{cases} 400 - x_1 - x_2 = x_3 \geq 0 \\ 300 - x_2 = x_5 \geq 0 \end{cases} \Rightarrow \begin{cases} x_2 \leq 400 \\ x_2 \leq 300 \end{cases}$$

未 tight 的 $\Rightarrow x_2 = 300 - x_5 \quad (0, 300, 100, 200, 0)$

变为

$$\begin{cases} 100 - x_1 + x_5 = x_3 \\ 200 - x_1 = x_4 \\ 300 - x_5 = x_2 \end{cases} \Rightarrow \begin{cases} x_1 \leq 100 \text{ 更紧} \\ \max x_1 = 100 - x_3 - x_5 \end{cases}$$

变为 $\max 1900 - \boxed{x_3 - 5x_5}$ 全负. 没有更好结果

Lecture 7 Graph

1. Graph decomposition

1) DFS (时间复杂度 $O(|V| + |E|)$)

① connectivity.

连通分量: each dfs 都找到一个. (任意 u, v 都有连接 path)

② previsit & postvisit.

如果引入 clock, 那么 $[pre(u), post(u)]$ & $[pre(v), post(v)]$

不可能相交 $\Rightarrow$ 栈先进后出

③ Types of edge.

(u, v)	{	a. Tree edge: part of DFS forest
$[u [v] u]$		b. Forward edge: 前面节点指向 nonchild 节点
$[v [u] v]$		c. back edges: 后面节点指向前节点
$[u] [v]$		d. cross edge: 不是祖先, 也不是后代

④ topologically sort.

$\hookrightarrow$ order the vertices s.t. every edge goes small to large

① 在 dag 中: 边 (u, v) , u 的 post num $>$ v 的.

② dag 中必有 1 source & a sink

find source
 $\hookrightarrow$ del & output

DFS 中的 $post(u) \geq post(v)$

$\Rightarrow u \rightarrow v$

$[[]] / [] []$

找 SCC 算法: ① G^R dfs 得 post num
② $G \leftarrow dfs$ 按 ① 中从大到小处理

⑤ connectivity for directed graphs. 每次 explore 就是找 SCC

将 SCC 合并为 meta node $\Rightarrow$ 最后整张图变为 dag.

最大 post number $\Leftarrow$ source / Reverse 后同样可找出 sink
最小 post 不是 sink

(2) BFS.

$\rightarrow$ 用 queue 实现, running time is $O(|V| + |E|)$

2. Shortest Path.

(1) def. $d(u, v) = \min \{ w(p) \mid p \text{ is path } u \text{ to } v \}$

$\rightarrow$ 子结构最优 & 满足三角不等式 $(d(u, v_2) \leq d(u, v_1) + d(v_1, v_2))$

(2) 单源最短

路径名字别弄错

① Dijkstra for nonnegative weight

从 S 出发

如果可整图.

先标记 $d(S) = 0, d(\text{others}) = \infty$

$\uparrow$ 加一起源点

二叉堆
 $O((m+n) \log n)$
Greedy

取最小 no 加入 $S = \{ \}$

检测负环:

更新 $d(\text{others})$: 重复即可

做完 $n-1$ 轮后
再松弛一遍 $\rightarrow$ 有负环

② Bellman-ford. $\rightarrow$ 可 early stop. #

定义 $OPT(i, v)$ 是 shortest $d(S, v)$, 用最多 i 条边.

$O(mn)$ time
 $O(n^2)$ space

$$OPT(i, v) = \begin{cases} 0 & , v = S \\ \infty & , v \neq S, i = 0 \\ \min \{ OPT(i-1, v), \min_{(u,v) \in E} \{ OPT(i-1, u) + w(u, v) \} \} \end{cases}$$

$\rightarrow$ 用 $M[u]$ 即可, 若 $M[u]$ 上一轮未更新, 得下

$$M[v] = \min (M[v], M[u] + w(u, v))$$

3) all-pair shortest paths

① 跑 n 遍 Bellman-Ford

$$\hookrightarrow \text{Time} = O(n^3 m)$$

$$\textcircled{2} \text{ 设 } d_{ij}^{(0)} = \begin{cases} 0 & i=j \\ \infty & i \neq j \end{cases}$$

$$\text{所有 } d_{ij}^{(m)} = \min_k \{ d_{ik}^{(m-1)} + a_{kj} \}$$

$\hookrightarrow$ 跑到 $m=n-1$ 即可 (再跑即可检查环)

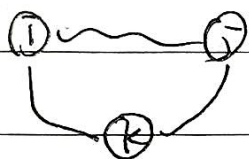
$\hookrightarrow$ 这个表达式可类如 matrix multiple

相同量 $\swarrow$

$$= \text{分可的 } O(n^3 \log n)$$

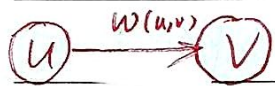
③ floyd-warshall 算法

$$C_{ij}^{(k)} = \min_k \{ C_{ij}^{(k-1)}, C_{ik}^{(k-1)} + C_{kj}^{(k-1)} \}$$



$$\hookrightarrow \text{Time} = O(n^3)$$

稀疏 $\swarrow$ ④ Johnson (加权到点上) 把负边权变成正的)



$$\hookrightarrow \tilde{w}(p) = w(p) + h(u) - h(v)$$

⑤ $\rightarrow$ 其他全为 0

$$\tilde{w}(u, v) = w(u, v)$$

① 先用 Bellman-Ford 跑算 $h(u), h(v)$ $O(mn)$

$$+ h(u) - h(v)$$

② reweight $\Rightarrow$ 再用 Dijkstra $O(n^2 \log n)$

$\downarrow$

v_i 到 v_k 的所有 Path

③ 重拍回去 $\rightarrow J = O(n^2)$

$$\text{由上知为 } O(mn + n^2 \log n)$$

加了 $|E|$ - 常量

②

$$h(v) = d(q, v)$$

重新 v 的

$$q \rightarrow \dots \rightarrow u \rightarrow v$$

$$\text{长度 } h(u) + w(u, v) \geq h(v)$$

$$\rightarrow \tilde{w}(p) \geq 0 \text{ 非负}$$

Lecture 8. Network Flow.

1. 概念定义.

1) ① 流网络 $G = (V, E, s, t, c)$

↳ flow 从 start 到 sink 的

② Cut: $s-t$ Cut 就是关于 G 的划分 (A, B) , $s \in A$ & $t \in B$

$$\text{cap}(A, B) = \sum_{A \rightarrow B} c(e) \Rightarrow \text{只算了出边}$$

↳ 最小割算这个

③ Flows: 是一个 function

$$\begin{cases} \text{capacity} & 0 \leq f(e) \leq c(e) \end{cases}$$

conservation 对于除 s, t 外所有, 出的 $f(e) =$ 入的 $f(e)$

$$|f| = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ into } s} f(e) \Rightarrow s \text{ 发出的净流量}$$

↳ 最大流算这个

2. Property:

1) Flow Value Lemma.

任意流 f , 任意 $s-t$ 割 (A, B) : f 的大小就是 $\text{cross}(A, B)$ 的

$$v(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e)$$

证明显然, 把 A 中打包, 只有 s 在流出

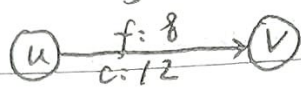
2) Duality:

任意流 f , 任意 $s-t$ 割 (A, B) , 如果两者相等

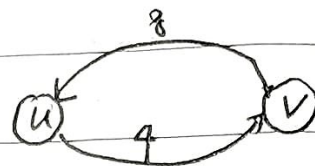
$$|v(f)| \leq \text{cap}(A, B) \Rightarrow \text{只出不进} \quad \text{即最大流最小割}$$

3. Ford-Fulkson Algorithm.

N) residual edge



变成



显然如果 $flow = capacity$ 直接反向

augmenting path: 有在 G_f 中 $s \sim t$ 的 path (即可增长掉)
bottleneck: An Path 中 最小的 residual capacity.

(2) Algorithm:

$\forall e, f(e) = 0$ 算 G_f .

找 Augu Path, 更新 f . 直到没有.

4. Max-Flow Min-Cut Theorem

(1) There exist a cut (A, B) s.t. $v(f) = cap(A, B)$

(2) Flow f is a max flow.

(3) There is no augmenting path

上面三个表述相等价.

$$\begin{aligned} (3) \Rightarrow (1): \quad v(f) &= \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ into } s} f(e) = \sum_{e \text{ out of } A} f(e) - 0 \\ &= cap(A, B). \end{aligned}$$

5. 时间复杂度:

(1) 定义最大容量为 C .

$\Rightarrow val(f^*) \leq n \times C \Rightarrow$ 再用 DFS/BFS 找边 $\Rightarrow O(mnc)$

4) 改进

2分去找先 mask 小于 $\frac{c}{2}, \frac{c}{4}, \dots$, 先插小池
Edmond-Karp 算法 $\Rightarrow$ Capacity - Scaling

$\rightarrow$ 时间复杂度

总阶段数 $O(\log C)$, 每个 scaling 时有 $O(m)$ argument $\Rightarrow$ 总共 $O(m^2 \log C)$ bfs/dfs

$\rightarrow$ 一个 Δ -phase 结束, 最大流量最多 $m\Delta$

每个 bottle neck 最多 $\Delta \Rightarrow$ 需要 $O(m)$

Lecture 9. Turing Machine

1. 单带 turning machine.

(1) 符号集: 还有一个读头 $\rightarrow$ 左端, 右端

$\{s_1, s_2, \dots, s_n\} \cup \{0, 1\} \cup \{\square\}$ of symbols

开始 $\{q_s, q_1, q_2, \dots, q_m, q_H\}$ 状态 $\rightarrow$ 结束

(2) instruction form:

$\langle \underline{q_i}, \underline{s_j} \rangle \rightarrow \langle \underline{q_k}, \underline{s_k}, L/R, S \rangle$ 写好之后设计

q_i 状态读到 s_j , 写成 s_k 变为 q_k 状态并移动.

2. 多带 turning machine.

(1) 符号集 同上 但 1-st tap is read-only

(2) instruct form: (2-tap)

$$\langle q_1, \square, \square \rangle \rightarrow \langle q, \square, R, R \rangle$$

q_1 状态 tap1, tap2 都是 $\square$, 转为 q , tap2 写为 $\square$. 两个头向右移

3. TM Variations

(1) Language System:

① $\Sigma = \{a_1, \dots, a_k\}$ is alphabet

② string is seq consist of a_i

③ $\Sigma^* = \{a, b\}^* = \{ \underbrace{\square}_{\text{空串}}, a, b, aa, bb, \dots \}$ finite

(2) ① $\rightarrow$ 无论语言多复杂, 都与 $\{0, 1, \square, \square\}$ 一样能力
只是 $4 \log |I|$ 倍 encode & decode

② k 带在 $T(n)$ 能做的
单带 只用 $\boxed{5kT(n)}$

$\rightarrow$ 先扫一遍得虚拟 $(k \times T(n))$, 再跑 $T(n)$.

③ bi directional TM 用 $T(n)$ 步

可以被普通单向用 $4T(n)$ 步 $\Rightarrow$ 对折, 用 $\underline{1}$ 表示上, $\underline{2}$ 表示下

都是 IN 上的, 要先转换到 IN.

4. 可计算和可判定

$$f^* = \lambda f \lambda^{-1}$$

(1) Turning - computable:

$$M(a_1, \dots, a_n) \downarrow b \text{ iff } f(a_1, \dots, a_n) = b$$

在 b 停机

可计算函数 $f \Leftrightarrow \exists$ 一台 TM M , 使得 M 计算 f .

(2) decision. (输出 yes/no)

如果这个 0-1 输出函数, turning-computable,
则是 decidable.

Lecture 10. NP Reduction

1. 定义

P 问题: decision problems 有一个多项式解法
对应一台 DTM

NP 问题: decision problems 有一个多项式时间验证器
对应一台 NTM

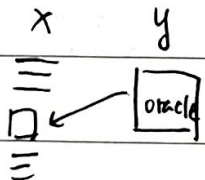
(2) poly - reduction:

$x \leq_p y$, x 多项式时间归约到 y .
somehow 代表归约

① Cook 归约:

$$x \leq_p y$$

x 用 y 的求解器



② Karp 归约:

构造一个 f , $\forall x \in X, f(x) \in Y$

证明: 1) $x \text{ yes} \Rightarrow y \text{ yes}$

2) $x \text{ yes} \Leftrightarrow y \text{ yes} \Rightarrow$ 常用逆否

y 是特殊的例子, 不是 $\forall$!

2. 经典例子及方法

(1) vertex cover $\equiv_p$ Independent set

证明一半, $IS \leq_p VC$

(i) $\forall x \in IS$, 即 $G=(V, E)$, $S \subseteq V$, $|S| \geq k$, 且其中任意两节点不连。

构造一个 $y \in VC$, 且 $G'=(V, E)$, $C' = V \setminus S$, $k' = n - k$, $|C'| \leq k'$

(ii) ① x yes $\Rightarrow$ 任意一条边, 至少有一个端点在 $V \setminus S$

$\Rightarrow |VC| = |V \setminus S| \leq k' \Rightarrow y$ is yes

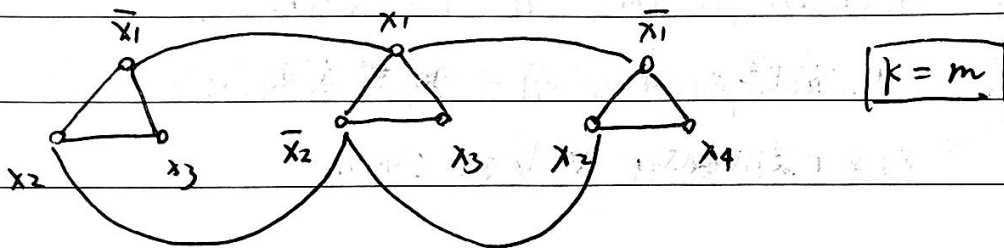
② y yes $\Rightarrow$ 至少有一个端点在 C . $\Rightarrow$ 不存在边两端在 $S = V \setminus C$

$\Rightarrow |S| = |V| - |C| \geq n - k' = k \Rightarrow x$ is yes

(2) 3-SAT $\leq_p$ IS

search $\leq_p$ decision

$$\phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$



NP-complete:

Y is NP, 且 $\forall x \in NP$, $x \leq_p Y$, the Y is NPC.

一般的方法:

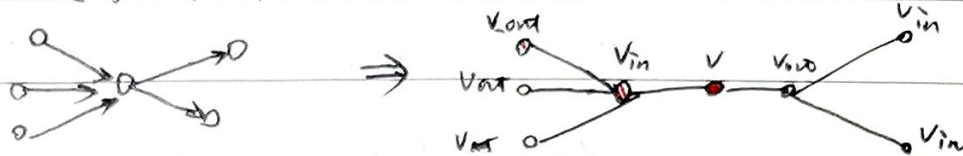
(1) 先证 Y is NP

(2) 选一个 NP-complete 问题 x . (如 3-SAT)

(3) 证明 $x \leq_p Y$

3) $3-SAT \leq_P DIR-HAM-CYCLE \leq_P HAM-CYCLE$

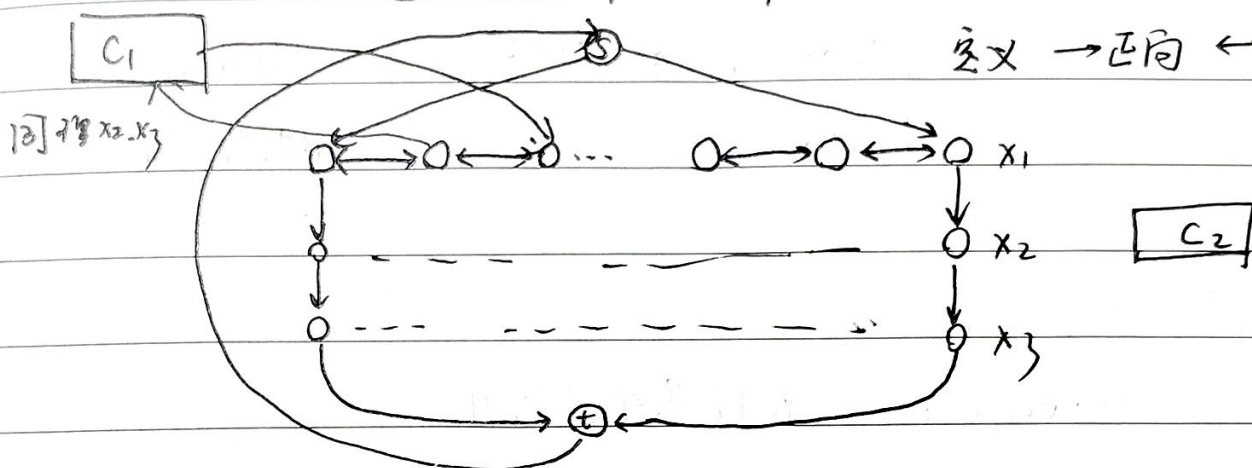
① 先构造后一个 $\leq_P$



② 再构造前一个 $\leq_P$

$$\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

正义 $\rightarrow$ 正向 $\leftarrow$ 负向



14) $HAM-CYCLE \leq_P LONGEST PATH$

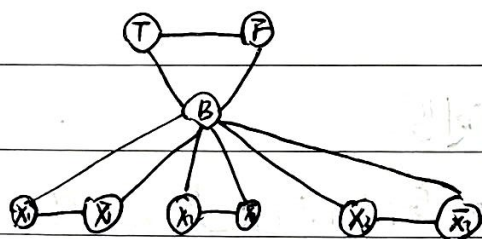
构造时直接去掉一条边, $k = |C| - 1$, f 直接设成最简圈

有 $n-1$ 条边的 path 到 n 个边 (反向或正)

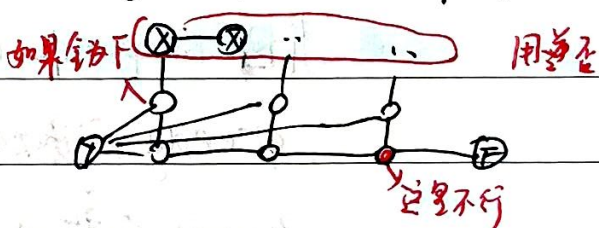
15) $3-SAT \leq_P 3-D MATCHING$ ($K=2$) 当 $K=3$ 为六边形...

$$\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

16) $3-SAT \leq_P 3-COLOR$



然后对每个 clause 作 gadget



17) $3-SAT \leq_P Subset-Sum$ k slides

Lecture 11. Approximation Algorithm.

1. Approximation Basic

1) 定义: ① $P = (I, sol, m, goal)$ $m(x, y) = \frac{goal(y, sol(x))}{goal(x, sol(x))}$

近似算法在保证性能的同时 不太坏

$$Appr\ ratio = \max \left\{ \frac{m(x, y)}{OPT}, \frac{OPT}{m(x, y)} \right\}$$

② $\rightarrow$ 主要分类: APX , $\log$ - APX , $poly$ - APX , exp - APX

③ $\left. \begin{array}{l} PTAS \\ FPTAS \end{array} \right\} \frac{A_g}{A^*} \leq 1 + \epsilon$, 时间复杂度可能不好
时间保持在一定范围多

2. Greedy Algorithm

1) recipe: 先按某个标准排序
从空集开始往里面加东西

2) example:

① Set cover (最小化开销)

greedy 的选择 $\frac{c(S)}{|S|-c|}$ 最小的 ($H(n)$ appr)

$\rightarrow$ 给元素编号按覆盖顺序 $e_1, e_2, \dots, e_k, \dots, e_n$

$$price(e_k) \leq \frac{m^*(u)}{|remain|} \leq \frac{m^*(u)}{n-k+1}$$

$$ALG = \sum_{k=1}^n price(e_k) \leq OPT \left(1 + \frac{1}{2} + \dots + \frac{1}{n} \right) = H_n \cdot OPT$$

② 背包 Knapsack 问题

greedy 的选择 单位价值最高的 $\rightarrow$ 可能误差
greedy 要么, 最大价值的, 要么单位价值高的

$$\frac{m^*}{m_H} < 2 \text{ 即 } m_H > \frac{1}{2} m^* \text{ 2 近似比}$$

$$m_H(X) = \max \{ p_{\max}, m_g(X) \}$$

设 a_j 是第一个未被放进去的物品. $P = \sum_{i=1}^{j-1} p_i$ $A = \sum_{i=1}^{j-1} a_i$

$$m^*(X) \leq P + (b-A) \frac{p_j}{a_j} < P + p_j$$

若 $p_j \leq P$, 且有 $m_g(A) \geq P$ 所以 $m^*(X) \leq 2P < 2m_g(X) \leq 2m_H(X)$

若 $P \leq p_j$, 且有 $p_{\max} \geq p_j$ 所以 $m^*(X) \leq 2p_j \leq 2p_{\max} \leq 2m_H(X)$ #

③ max independent set problem

greedy 是选 degree 最少, 删邻居和自己, 再选

→ 证 $\frac{m^*(G)}{m_g(G)} \leq 1 + \frac{m}{n}$ 证明欣赏 slides 即可

3 Sequential Algorithm

(1) 定义: 按某顺序处理, 一旦处理就不能动.

例 example:

① Max-Cut.

greedy 算法同数学基础的

$$\hookrightarrow \frac{OPT}{A_g} \leq 2 \quad (2 \text{ 近似比})$$

$$A_g \geq \frac{E}{2} \geq \frac{OPT}{2} \quad \#$$

② job-scheduling on p -machines

greedy 算法 放在负载最小机器上 (先长后短)

$$\hookrightarrow \frac{m_L(T)}{m^*(T)} \leq \frac{4}{3} - \frac{1}{3p}$$

$\hookrightarrow$ 当 $\frac{m^*(T)}{3}$ 时最多 2 倍

$$p < |T| \leq 2p \quad (\text{补 } 0 \text{ 到 } 2p)$$

$$m^*(T) \geq \max l_i$$

$$m^*(T) \geq \frac{\sum l_i}{p}$$



大和大更差

$$\Rightarrow m_L(T) = m^*(T)$$

$\frac{1}{3} l_{\min} \leq \frac{m^*(T)}{3}$ 时.

用反证. 假设反例 J. 最后将 J_j 在 $m_L(T)$ 的机器上.

之前 $A_h \leq \frac{W-l_j}{p}$ $\frac{W}{p} \leq m^*(T)$ $l_j = l_{\min} \leq \frac{m^*(T)}{3}$

$$m_L(T) = A_h + l_j \leq \frac{W-l_j}{p} + l_j = \frac{W}{p} + (1-\frac{1}{p})l_j$$

$$\leq (\frac{4}{3} - \frac{1}{3p}) m^*(T)$$

4. Local Search

(1) 定义: 先有某一可行解, 然后在邻居空间来优化.

(2) example:

① 并行机调度问题

→ 每次看最后完成的 job, 尝试放到更空的 machine

2-Approximation: $C_{max} \leq 2C^*_{max}$

$$\begin{cases} p_i \leq \max_j p_j \leq C^*_{max} \\ \frac{p}{n} \leq C^*_{max} \end{cases} \quad \begin{aligned} C_i &= \sum_{j \in J_i} p_j \\ T_i &\geq S_i \Rightarrow S_i \leq \frac{p}{n} \end{aligned}$$

② Max-Cut 问题

→ 对每个点尝试移到更大的 cut

$$\frac{1}{2} m^*(G) \leq m_N(G)$$

$$m = \sum_{v \in V} m_1 + \sum_{v \in V} m_2 + m_N(G)$$

$$|m_1| - |m_2| \leq 0 \Rightarrow \text{对 } v_1, v_2 \text{ 有 } 2m_1 \leq m_N(G)$$

$$2m_2 \leq m_N(G)$$

$$\dots \Rightarrow m_N(G) \geq \frac{1}{2} m > \frac{1}{2} m^*(G)$$

5. LP-Rounding

1) 设 $X =$ ILP 问题
 $x_i \in \{0, 1\}$

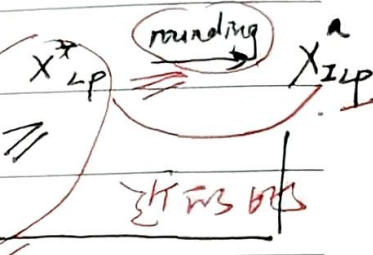
最优化问题时

relaxation

LP
 $x_i \geq 0$

solver

与 min. max 有关



(K) Set-Cover

$X_S = \begin{cases} 1, & \text{选择集合 } S \\ 0, & \text{不选} \end{cases}$

$$\min \sum C(S) \cdot X_S$$

relaxation

$$\text{s.t. } \sum X_S \geq 1$$

rounding

$f =$ 出现最多次数

$$\min \sum C(S) \cdot X_S$$

$$X_S \geq 0$$

用 $\frac{1}{f}$ rounding

$$\sum_{S \in \mathcal{S}} X_S \geq 1$$

$S \in \mathcal{S}$

$$X_{LP}^* = \frac{1}{2} \text{ (slides 308)}$$

这个值记

显然是一种 set-cover, S' $C(S) \leq f \cdot C(S) X_S$

$$\text{cost}(S') \leq f \sum_{S \in \mathcal{S}} C(S) X_S \leq f \sum_{S \in \mathcal{S}} C(S) X_S = f \cdot \text{OPT}_f \leq f \cdot \text{OPT}$$

random rounding.

① n 个概率 X_S 选 S , $E(\text{cost}(S')) = \sum_{S \in \mathcal{S}} X_S C_S = \text{OPT}_f$

$$e_i \text{ 在 } k \text{ 个集合 } P(\text{元素 } e_i \text{ 未覆盖}) = \prod_{j=1}^k (1 - X_{S_j}) \leq \left(\frac{1 - X_{S_j}}{k} \right)^k = \left(\frac{k-1}{k} \right)^k \leq \frac{1}{e}$$

② 重复① $c \log n$ 次, 然后 union 起来

$$\text{这里 } \left(\frac{1}{e} \right)^{c \log n} \leq \frac{1}{4n} \quad P(C' \text{ is not valid}) \leq n \cdot \frac{1}{4n} = \frac{1}{4}$$

$$E(\text{cost}(C')) \leq c \log n \cdot \text{OPT}_f$$

$$P(\text{cost}(C') \geq 4c \log n \cdot \text{OPT}_f) \leq \frac{E(\text{cost}(C'))}{4c \log n \cdot \text{OPT}_f} = \frac{1}{4}$$

两个坏事件都发生最多 $\frac{1}{2}$.